

EEA

(Winter Project)

FINAL REPORT

(Group 10)

Team Leader:

Pranesh S (240771)

Team Members:

Arman Kumar Singh (240183)

Nipun Nandwani (240700)

Uday Charan (240743)

Praveen Siddarth (240552)

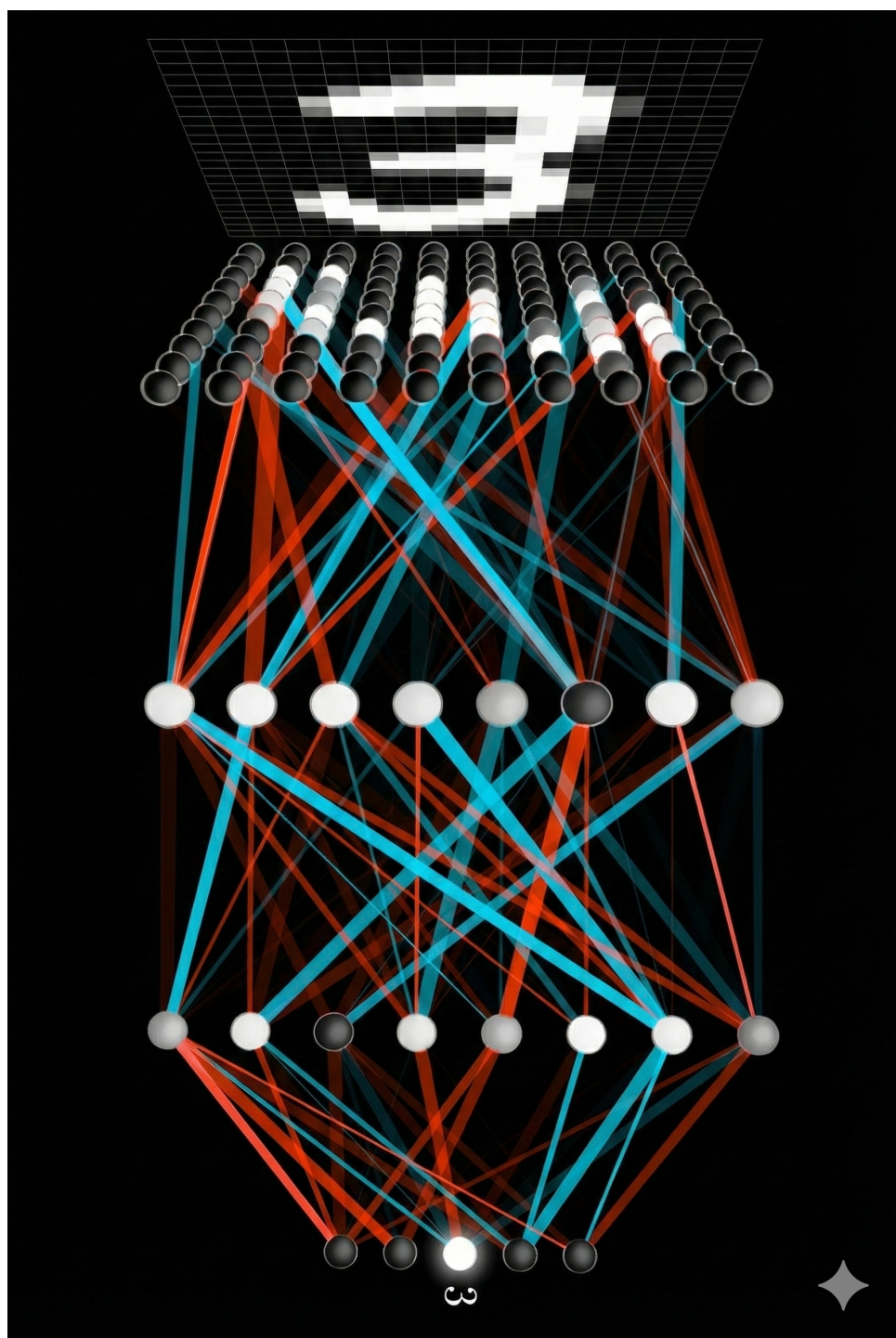
Sherwin Leo M.L (240970)

Vaibhav Gupta (241124)

Sandeep Kumar Gupta (240928)

Anubhav Jha (240152)

DEEP NEURAL NETWORK COMPRESSION & CV



Contents

1	Introduction	5
2	Digital Image Representation	5
2.1	What is a Digital Image?	5
2.2	Grayscale Images	5
2.3	Color Images	5
2.4	Images as Mathematical Objects	5
3	Image Loading and Color Ordering	6
4	Grayscale Conversion	6
5	Frequency Domain Analysis	6
5.1	Motivation	6
5.2	Fourier Transform	6
5.3	Magnitude and Phase	7
5.4	DFT vs FFT	7
6	Frequency Domain Filtering	7
6.1	Low-Pass Filtering	7
6.2	High-Pass Filtering	7
7	Histograms and Intensity Distribution	8
8	Color Spaces	8
8.1	HSV Color Space	8
8.2	HSL Color Space	8
9	Image Adjustments	8
9.1	Brightness	8
9.2	Contrast	9
9.3	Gamma Correction	9
10	Convolution and Filtering	9
11	Blurring Techniques	9
11.1	Average Blur	9
11.2	Gaussian Blur	9
12	Gradients and Edge Detection	9
12.1	Sobel Operator	10
12.2	Canny Edge Detector	10

13 Sharpening Techniques	10
13.1 Laplacian Sharpening	10
13.2 Unsharp Masking	10
14 Corner Detection	10
15 Hough Transform for Parametric Shape Detection	10
15.1 Motivation and Core Idea	11
15.2 Line Detection Using the Hough Transform	11
15.2.1 Parametric Representation of a Line	11
15.2.2 Voting in Parameter Space	11
15.2.3 Algorithm Steps for Line Detection	11
15.2.4 Properties and Advantages	12
15.3 Circle Detection Using the Hough Transform	12
15.3.1 Circle Parametrization	12
15.3.2 Voting Strategy	12
15.3.3 Algorithmic Challenges	12
15.4 Ellipse Detection	13
15.5 Robustness and Practical Considerations	13
16 Feature Extraction for Image-Based Classification	13
16.1 Color Features	14
16.1.1 Mean Hue	14
16.1.2 Standard Deviation of Hue	14
16.1.3 Mean and Standard Deviation of Saturation	14
16.1.4 Mean and Standard Deviation of Value	14
16.2 Shape Features	14
16.2.1 Area Ratio	15
16.2.2 Aspect Ratio	15
16.2.3 Solidity	15
16.2.4 Circularity	15
16.3 Moment-Based Shape Descriptors: Hu Moments	15
16.3.1 Image Moments	15
16.3.2 Hu Moments	16
16.4 Texture Features: Local Binary Patterns (LBP)	16
16.4.1 Conceptual Idea	16
16.4.2 LBP Encoding	16
16.4.3 Properties of LBP	16
16.5 Complementarity of Features	17
17 Machine Learning	17

18 Linear Regression	17
18.1 Model Representation	17
18.2 Loss Function	18
18.3 Optimization via Gradient Descent	18
19 Logistic Regression	18
19.1 Motivation	18
19.2 Model and Sigmoid Function	18
19.3 Binary Cross-Entropy Loss	18
19.4 Parameter Updates	19
20 Unsupervised Learning: K-Means Clustering	19
20.1 Objective	19
20.2 Algorithm	19
20.3 Convergence	19
21 Regularization and Ridge Regression (L2)	19
21.1 Overfitting	19
21.2 L2 Regularization	19
22 Multi-Layer Perceptron (MLP)	20
22.1 Motivation and Limitations of Linear Models	20
22.2 Network Architecture	20
22.3 Forward Propagation	20
22.4 Loss Function	21
22.5 Backpropagation	21
22.6 Parameter Updates	21
23 Vanilla Neural Networks	21
24 Convolutional Neural Networks (CNNs)	21
24.1 Motivation	22
24.2 Convolution Operation	22
24.3 Key Properties of CNNs	22
24.4 Non-Linearity and Pooling	22
24.5 CNN Architecture	22
24.6 Backpropagation in CNNs	23
24.7 Advantages of CNNs	23
25 PyTorch: Tensors, Data, and Gradients	23
25.1 Tensors	23
25.2 GPU Acceleration	23
25.3 Automatic Differentiation (Autograd)	24
25.4 Backward Pass	24
25.5 Gradient Control	24

26 Datasets and DataLoaders in PyTorch	24
26.1 Dataset Class	24
26.2 DataLoader	24
27 Deep Neural Network Compression Pipeline	24
27.1 Overview	24
27.2 Pipeline Architecture	25
27.3 Model Architecture	25
27.4 Weight Pruning	25
27.4.1 Effect of Pruning	25
27.5 Weight Quantization	26
27.5.1 Advantages	26
27.6 Model Serialization	26
27.7 Entropy-Based Compression	26
27.8 Training and Compression Strategy	26
27.9 Performance Metrics	27
27.10 Observations	27
28 Results and Analysis	27
28.1 Baseline Model Performance	27
28.2 Effect of Pruning	28
28.3 Effect of Quantization	28
28.4 Compressed Model Performance	28
28.5 Storage and Compression Analysis	28
28.6 Memory Usage	29
28.7 Overall Insights	29
29 Conclusion	29
30 Team Contributions	30

1 Introduction

This report presents a comprehensive summary of the concepts and methodologies covered in the project up to the present stage. It systematically documents the theoretical foundations, mathematical formulations, and algorithmic principles related to CV and machine learning that were discussed during the sessions.

The report is intended to consolidate the material taught so far by providing clear explanations alongside formal mathematical expressions, reflecting both the structured content of the sessions and the detailed conceptual discussions delivered in class. Emphasis is placed on clarity, completeness, and coherence to demonstrate a thorough understanding of the topics covered.

2 Digital Image Representation

2.1 What is a Digital Image?

A digital image is a sampled and quantized representation of a continuous visual scene. In computational terms, an image is treated as a matrix whose elements correspond to pixel intensity values.

2.2 Grayscale Images

A grayscale image is represented as a two-dimensional matrix:

$$I \in \mathbb{R}^{H \times W}$$

where H and W denote the height and width of the image. Each matrix entry $I(x,y)$ represents the intensity of light at pixel location (x,y) .

2.3 Color Images

Color images typically use the RGB color model and are represented as:

$$I \in \mathbb{R}^{H \times W \times 3}$$

Each pixel is defined as:

$$I(x,y) = (R(x,y), G(x,y), B(x,y))$$

where each channel value lies in the range $[0, 255]$.

For mathematical operations, pixel values are normalized:

$$I_f = \frac{I}{255}$$

This normalization prevents numerical instability and ensures compatibility with floating-point computations.

2.4 Images as Mathematical Objects

Treating images as matrices allows the application of:

- Linear algebra
- Signal processing
- Fourier analysis
- Convolution operations

This mathematical viewpoint forms the backbone of computer vision.

3 Image Loading and Color Ordering

Different image processing libraries adopt different color channel conventions. For example, OpenCV stores images in BGR format, whereas most visualization libraries expect RGB ordering.

To ensure correctness:

$$I_{RGB} = \text{convert}(I_{BGR})$$

Failure to perform this conversion results in incorrect color interpretation, which can propagate errors throughout the processing pipeline.

4 Grayscale Conversion

Grayscale conversion reduces a color image to a single intensity channel while preserving perceptual luminance.

$$I_{gray}(x, y) = 0.299R(x, y) + 0.587G(x, y) + 0.114B(x, y)$$

These weights correspond to human visual sensitivity, where the eye is most sensitive to green light and least sensitive to blue.

Grayscale images are widely used because:

- They reduce computational complexity
- Many structural features do not require color
- Frequency-domain and edge operations become simpler

5 Frequency Domain Analysis

5.1 Motivation

Spatial-domain analysis examines pixel values directly. However, many image properties such as texture, smoothness, and noise are better understood in the frequency domain.

5.2 Fourier Transform

The 2D Discrete Fourier Transform (DFT) is defined as:

$$F(u, v) = \sum_{x=0}^{H-1} \sum_{y=0}^{W-1} I(x, y) e^{-j2\pi(\frac{ux}{H} + \frac{vy}{W})}$$

This transform decomposes an image into a weighted sum of sinusoidal basis functions.

5.3 Magnitude and Phase

The Fourier transform produces:

- **Magnitude Spectrum** $|F(u, v)|$: strength of frequency components
- **Phase Spectrum** $\angle F(u, v)$: spatial alignment of structures

An important observation is that phase information preserves most of the structural content of an image, while magnitude captures textural information.

5.4 DFT vs FFT

The naive computation of DFT has complexity $O(N^2)$, making it impractical for large images. The Fast Fourier Transform (FFT) reduces complexity to:

$$O(N \log N)$$

This makes frequency-domain processing feasible in real-world applications.

6 Frequency Domain Filtering

Low frequencies correspond to smooth intensity variations, whereas high frequencies correspond to edges and noise.

6.1 Low-Pass Filtering

Low-pass filtering retains low-frequency components:

$$F_{LPF} = F \cdot M_{LPF}$$

This results in image blurring and noise reduction.

6.2 High-Pass Filtering

High-pass filtering retains high-frequency components:

$$F_{HPF} = F \cdot M_{HPF}$$

This enhances edges and fine details.

Reconstruction is performed using the inverse Fourier transform:

$$I = |\mathcal{F}^{-1}(F)|$$

7 Histograms and Intensity Distribution

A histogram describes how pixel intensities are distributed across an image:

$$H(i) = \#\{(x,y) \mid I(x,y) = i\}$$

Histograms provide insight into:

- Exposure
- Brightness
- Contrast

RGB histograms further help identify color imbalance and dominant hues.

8 Color Spaces

8.1 HSV Color Space

HSV separates color perception into:

- Hue (H): dominant wavelength
- Saturation (S): color purity
- Value (V): brightness

This separation aligns more closely with human vision.

8.2 HSL Color Space

HSL replaces Value with Lightness:

$$L = \frac{\max(R, G, B) + \min(R, G, B)}{2}$$

Saturation behaves differently in HSL, peaking at $L = 0.5$.

9 Image Adjustments

9.1 Brightness

Brightness is adjusted by adding a constant:

$$I' = I + \beta$$

This shifts the histogram but does not increase contrast.

9.2 Contrast

Contrast scaling:

$$I' = \alpha I$$

Values of α control intensity spread.

9.3 Gamma Correction

Human vision is nonlinear. Gamma correction compensates for this:

$$I_{out} = I_{in}^{1/\gamma}$$

where $\gamma \approx 2.2$.

10 Convolution and Filtering

Convolution replaces each pixel with a weighted sum of its neighbors:

$$(I * K)(x, y) = \sum_i \sum_j I(x + i, y + j) K(i, j)$$

Kernel properties determine behavior:

- Blur kernels sum to 1
- Edge detectors sum to 0
- Sharpening kernels amplify differences

11 Blurring Techniques

11.1 Average Blur

All neighboring pixels contribute equally, resulting in uniform smoothing but loss of detail.

11.2 Gaussian Blur

Gaussian blur uses:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

This produces natural-looking blur and preserves edges better.

12 Gradients and Edge Detection

Gradients measure intensity change:

$$|\nabla I| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$$

Edges correspond to high gradient magnitude.

12.1 Sobel Operator

Approximates gradients using directional convolution kernels.

12.2 Canny Edge Detector

Pipeline:

1. Gaussian smoothing
2. Gradient computation
3. Non-maximum suppression
4. Double thresholding
5. Edge tracking by hysteresis

Canny is considered the gold standard classical edge detector.

13 Sharpening Techniques

13.1 Laplacian Sharpening

Uses second-order derivatives:

$$I_{sharp} = I - \alpha \nabla^2 I$$

13.2 Unsharp Masking

Enhances edges by:

$$I_{sharp} = I + k(I - B)$$

where B is a blurred version of the image.

14 Corner Detection

Corners are points with intensity variation in all directions. They are crucial for feature matching and object recognition.

15 Hough Transform for Parametric Shape Detection

Detecting geometric primitives such as lines and circles is a fundamental task in computer vision. While edge detection identifies boundary pixels, it does not directly provide a global geometric interpretation. The Hough Transform addresses this limitation by detecting shapes through a voting mechanism in a parameter space, making it highly robust to noise, occlusion, and broken edges.

15.1 Motivation and Core Idea

In real images, edges belonging to a single geometric structure may be discontinuous or corrupted by noise. Classical local methods often fail under such conditions. The Hough Transform overcomes this by converting the problem of detecting shapes in the image space into a peak detection problem in a parameter space.

The key idea is that points lying on the same geometric shape will vote for the same parameters describing that shape.

15.2 Line Detection Using the Hough Transform

15.2.1 Parametric Representation of a Line

A straight line can be represented in slope-intercept form:

$$y = mx + c$$

However, this representation becomes unstable for vertical lines where the slope m tends to infinity. To avoid this issue, the Hough Transform uses the normal form:

$$x \cos \theta + y \sin \theta = \rho$$

where:

- ρ is the perpendicular distance of the line from the origin
- θ is the angle of the normal to the line

This representation uniquely describes all possible lines in a bounded parameter space.

15.2.2 Voting in Parameter Space

Each edge pixel (x, y) corresponds to infinitely many lines passing through it. For each possible angle θ , a corresponding ρ is computed:

$$\rho = x \cos \theta + y \sin \theta$$

These (ρ, θ) pairs are accumulated in a 2D array called the *accumulator*. Each edge pixel casts votes along a sinusoidal curve in the parameter space.

15.2.3 Algorithm Steps for Line Detection

1. Perform edge detection (typically using the Canny detector)
2. Initialize an accumulator array over discretized (ρ, θ) values
3. For each edge pixel:
 - For each θ , compute ρ
 - Increment the corresponding accumulator cell

4. Identify peaks in the accumulator
5. Map the detected parameters back to lines in image space

Peaks in the accumulator correspond to prominent lines in the image.

15.2.4 Properties and Advantages

The Hough Transform for lines:

- Is robust to noise and gaps in edges
- Can detect multiple lines simultaneously
- Works well even when edges are partially missing

However, the method requires careful discretization and threshold selection.

15.3 Circle Detection Using the Hough Transform

15.3.1 Circle Parametrization

A circle with center (a, b) and radius r is defined as:

$$(x - a)^2 + (y - b)^2 = r^2$$

This representation introduces three parameters, making the accumulator three-dimensional.

15.3.2 Voting Strategy

For each edge pixel (x, y) :

- Assume a radius r
- Vote for all possible centers (a, b) satisfying the equation

In practice, the radius range is constrained to reduce computational complexity.

15.3.3 Algorithmic Challenges

Circle detection is computationally expensive due to the higher dimensionality of the parameter space. Common optimizations include:

- Gradient-based voting (restricting center direction)
- Radius range limitation
- Probabilistic Hough Transform

Despite the cost, the method remains highly effective for detecting circular structures.

15.4 Ellipse Detection

Ellipse detection extends the Hough Transform further. An ellipse is described by five parameters:

$$(a, b, R_x, R_y, \theta)$$

Due to the high-dimensional parameter space, direct ellipse detection is computationally expensive and often replaced by approximation or learning-based methods in practice.

15.5 Robustness and Practical Considerations

The Hough Transform is particularly robust because:

- Each pixel contributes independently
- Noise pixels distribute votes randomly
- True shapes produce coherent peaks

However, limitations include:

- High memory usage
- Sensitivity to parameter discretization
- Increased complexity for higher-dimensional shapes

Despite these challenges, the Hough Transform remains a cornerstone algorithm in classical computer vision due to its conceptual simplicity and robustness.

16 Feature Extraction for Image-Based Classification

Raw images, although rich in visual information, are not directly suitable for classical machine learning models. Feature extraction aims to transform an image into a compact numerical representation that captures the most discriminative characteristics such as color, shape, and texture. These features must ideally be robust to scale, rotation, illumination changes, and noise.

In this course, three major categories of handcrafted features were studied:

- Color-based features
- Shape-based features
- Texture-based features

Each category captures complementary information, and their combination leads to a more expressive and discriminative representation.

16.1 Color Features

Color features describe the chromatic properties of an object and are especially useful when color plays a significant role in object identity, such as in fruit classification.

To decouple color from illumination effects, images are converted from RGB space to HSV space. The HSV representation aligns better with human color perception by separating chromatic information (Hue and Saturation) from intensity (Value).

16.1.1 Mean Hue

The mean hue represents the dominant color of the object:

$$\mu_H = \frac{1}{N} \sum_{i=1}^N H_i$$

This feature is particularly effective in distinguishing objects with distinct color identities (e.g., bananas vs apples).

16.1.2 Standard Deviation of Hue

$$\sigma_H = \sqrt{\frac{1}{N} \sum_{i=1}^N (H_i - \mu_H)^2}$$

This measures the uniformity of color distribution. Fresh and uniform objects exhibit low variance, while spotted or multi-colored objects exhibit higher variance.

16.1.3 Mean and Standard Deviation of Saturation

Mean saturation reflects color purity, while its variance captures surface irregularities:

$$\mu_S = \frac{1}{N} \sum S_i, \quad \sigma_S = \sqrt{\frac{1}{N} \sum (S_i - \mu_S)^2}$$

16.1.4 Mean and Standard Deviation of Value

These features capture overall brightness and lighting consistency:

$$\mu_V = \frac{1}{N} \sum V_i, \quad \sigma_V = \sqrt{\frac{1}{N} \sum (V_i - \mu_V)^2}$$

Together, the six color features provide a compact and robust chromatic description.

16.2 Shape Features

Shape features describe the geometric structure of an object and are extracted from the object's contour, typically obtained using edge detection.

16.2.1 Area Ratio

The area ratio measures the relative size of the object:

$$\text{Area Ratio} = \frac{\text{Contour Area}}{\text{Image Area}}$$

This feature is scale-invariant and helps distinguish objects of different physical sizes.

16.2.2 Aspect Ratio

Aspect ratio describes elongation:

$$\text{Aspect Ratio} = \frac{w}{h}$$

Elongated objects such as bananas exhibit high aspect ratios, whereas round objects such as apples approach unity.

16.2.3 Solidity

Solidity measures how compact an object is:

$$\text{Solidity} = \frac{\text{Contour Area}}{\text{Convex Hull Area}}$$

Objects with smooth, convex shapes have high solidity, while irregular or indented shapes have lower solidity. This feature is robust to small surface noise.

16.2.4 Circularity

Circularity quantifies how close an object is to a perfect circle:

$$\text{Circularity} = \frac{4\pi A}{P^2}$$

A value of 1 indicates a perfect circle, making this feature particularly effective for distinguishing round and elongated objects.

16.3 Moment-Based Shape Descriptors: Hu Moments

16.3.1 Image Moments

Image moments are statistical measures that describe the spatial distribution of pixel intensities, analogous to moments of inertia in physics.

The *raw moments* are defined as:

$$m_{pq} = \sum_x \sum_y x^p y^q I(x, y)$$

To remove translation dependency, *central moments* are computed:

$$\mu_{pq} = \sum_x \sum_y (x - \bar{x})^p (y - \bar{y})^q I(x, y)$$

where $(\bar{x}, \bar{y})$ is the centroid of the object.

16.3.2 Hu Moments

Hu derived seven invariant combinations of normalized central moments:

$$\phi_1, \phi_2, \dots, \phi_7$$

These moments are invariant to:

- Translation
- Scale
- Rotation

In practice, lower-order Hu moments are more stable and less sensitive to noise. Hence, only the first two moments are used:

- ϕ_1 : overall shape spread
- ϕ_2 : shape variance

Higher-order moments capture finer details but are often noisy and less reliable.

16.4 Texture Features: Local Binary Patterns (LBP)

Texture describes the local spatial arrangement of pixel intensities and is essential for distinguishing objects with similar shapes but different surface properties.

16.4.1 Conceptual Idea

Local Binary Patterns (LBP) encode texture by comparing a pixel with its neighbors. For a center pixel g_c and its neighbors g_p :

$$s(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

16.4.2 LBP Encoding

The LBP code is computed as:

$$LBP = \sum_{p=0}^{P-1} s(g_p - g_c) 2^p$$

Each pixel is thus assigned a binary code representing its local texture pattern.

16.4.3 Properties of LBP

LBP features are:

- Robust to illumination changes

- Computationally efficient
- Optionally rotation-invariant

To obtain a global texture descriptor, a histogram of LBP codes is computed over the image.

16.5 Complementarity of Features

Color, shape, and texture features capture different aspects of an object. Individually, each feature type is limited, but together they provide a rich and discriminative representation.

This multi-feature strategy significantly improves classification performance compared to relying on any single feature type.

17 Machine Learning

Machine Learning (ML) is a subfield of artificial intelligence that focuses on designing algorithms that improve their performance on a task through experience. Instead of explicitly programming decision rules, machine learning models learn patterns directly from data by optimizing a well-defined objective function.

Formally, given a dataset:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$$

the goal of machine learning is to learn a function:

$$f_{\theta} : \mathbf{x} \rightarrow y$$

parameterized by θ , such that predictions generalize well to unseen data.

Machine learning models can be broadly categorized into:

- Supervised learning
- Unsupervised learning
- Deep learning

The following sections describe these paradigms in increasing order of complexity.

18 Linear Regression

18.1 Model Representation

Linear regression models the relationship between an input vector $\mathbf{x} \in \mathbb{R}^d$ and a continuous output $y \in \mathbb{R}$ as:

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b$$

where $\mathbf{w}$ represents the model weights and b is the bias term.

18.2 Loss Function

To quantify prediction error, Mean Squared Error (MSE) is used:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

This loss penalizes large deviations more heavily, encouraging accurate predictions.

18.3 Optimization via Gradient Descent

To minimize the loss, gradients are computed:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}}, \quad \frac{\partial \mathcal{L}}{\partial b}$$

Parameters are updated iteratively using gradient descent:

$$\begin{aligned} \mathbf{w} &\leftarrow \mathbf{w} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}} \\ b &\leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b} \end{aligned}$$

where η is the learning rate. This iterative update gradually moves parameters toward the minimum of the loss surface.

19 Logistic Regression

19.1 Motivation

Logistic regression extends linear models to binary classification problems by predicting probabilities instead of continuous values.

19.2 Model and Sigmoid Function

The linear score is computed as:

$$z = \mathbf{w}^T \mathbf{x} + b$$

This score is transformed using the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The output $\hat{y} = \sigma(z)$ represents the probability of belonging to the positive class.

19.3 Binary Cross-Entropy Loss

The loss function is:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

This loss is derived from maximum likelihood estimation of a Bernoulli distribution.

19.4 Parameter Updates

Gradients are computed with respect to weights and bias, and parameters are updated using gradient descent in the same manner as linear regression.

Logistic regression learns a linear decision boundary in feature space.

20 Unsupervised Learning: K-Means Clustering

20.1 Objective

K-Means is an unsupervised clustering algorithm that partitions data into K clusters by minimizing intra-cluster variance:

$$\min_{\{\mu_k\}} \sum_{i=1}^N \min_k \|\mathbf{x}_i - \mu_k\|^2$$

20.2 Algorithm

1. Initialize K cluster centers
2. Assign each data point to the nearest center
3. Recompute centers as cluster means
4. Repeat until convergence

20.3 Convergence

The algorithm monotonically decreases the objective function and converges to a local minimum.

21 Regularization and Ridge Regression (L2)

21.1 Overfitting

Overfitting occurs when a model memorizes training data rather than learning general patterns.

21.2 L2 Regularization

Ridge regression adds a penalty term:

$$\mathcal{L}_{\text{ridge}} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \lambda \|\mathbf{w}\|_2^2$$

This discourages large weights and improves generalization.

22 Multi-Layer Perceptron (MLP)

22.1 Motivation and Limitations of Linear Models

Linear models such as linear and logistic regression assume that the relationship between input features and output labels can be expressed as a linear function. However, many real-world problems involve highly non-linear relationships that cannot be captured by a single linear transformation.

The Multi-Layer Perceptron (MLP) overcomes this limitation by stacking multiple linear transformations interleaved with non-linear activation functions, enabling the model to learn complex decision boundaries.

22.2 Network Architecture

An MLP consists of:

- An input layer
- One or more hidden layers
- An output layer

Each layer performs an affine transformation followed by a non-linear activation.

22.3 Forward Propagation

For the l^{th} layer, the forward pass is defined as:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = \phi(\mathbf{z}^{(l)})$$

where:

- $\mathbf{W}^{(l)}$ is the weight matrix
- $\mathbf{b}^{(l)}$ is the bias vector
- $\phi(\cdot)$ is a non-linear activation function

A commonly used activation function is ReLU:

$$\phi(x) = \max(0, x)$$

Non-linearity is essential; without it, multiple layers would collapse into a single linear transformation.

22.4 Loss Function

For classification tasks, the network output is often passed through a softmax function:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

The corresponding loss is the categorical cross-entropy:

$$\mathcal{L} = -\sum_i y_i \log(\hat{y}_i)$$

This loss encourages the predicted probability of the correct class to be maximized.

22.5 Backpropagation

Training an MLP requires computing gradients of the loss with respect to all parameters. This is achieved using backpropagation, which applies the chain rule recursively.

For a given layer:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \delta^{(l)} (\mathbf{a}^{(l-1)})^T$$

where the error term $\delta^{(l)}$ is given by:

$$\delta^{(l)} = \left(\mathbf{W}^{(l+1)T} \delta^{(l+1)} \right) \odot \phi'(\mathbf{z}^{(l)})$$

This formulation shows how error propagates backward from the output layer to earlier layers.

22.6 Parameter Updates

Using gradient descent, parameters are updated as:

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}}$$

This iterative process gradually minimizes the loss.

23 Vanilla Neural Networks

Vanilla neural networks extend MLPs with multiple hidden layers:

$$\mathbf{a}^{(l)} = \phi(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$$

They are universal function approximators but become computationally expensive for high-dimensional inputs.

24 Convolutional Neural Networks (CNNs)

24.1 Motivation

Fully connected networks treat each input dimension independently and ignore spatial relationships. For image data, this leads to a massive number of parameters and poor generalization.

Convolutional Neural Networks (CNNs) are specifically designed to exploit the spatial structure of data by using local connectivity and weight sharing.

24.2 Convolution Operation

A convolutional layer applies a learnable kernel $\mathbf{K}$ over an input feature map $\mathbf{X}$:

$$(\mathbf{X} * \mathbf{K})(i, j) = \sum_m \sum_n \mathbf{X}(i + m, j + n) \mathbf{K}(m, n)$$

This operation extracts local patterns such as edges, corners, and textures.

24.3 Key Properties of CNNs

CNNs possess several important properties:

- **Local Receptive Fields:** neurons connect only to local regions
- **Weight Sharing:** same kernel applied across the image
- **Translation Equivariance:** shifts in input produce shifts in output

These properties dramatically reduce the number of parameters compared to fully connected layers.

24.4 Non-Linearity and Pooling

After convolution, a non-linear activation (usually ReLU) is applied:

$$\phi(x) = \max(0, x)$$

Pooling layers reduce spatial resolution:

$$y = \max_{(i,j) \in \Omega} x_{i,j}$$

Pooling introduces robustness to small spatial variations and reduces computation.

24.5 CNN Architecture

A typical CNN follows the pattern:

1. Convolution
2. Activation
3. Pooling
4. Repetition of above layers

5. Fully connected layers

Early layers capture low-level features, while deeper layers learn abstract representations.

24.6 Backpropagation in CNNs

Backpropagation in CNNs follows the same principles as MLPs but includes convolutional operations.

Gradients with respect to kernels are computed as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{K}} = \mathbf{X} * \delta$$

where δ is the error propagated from the next layer.

Weight sharing ensures that gradients from all spatial locations contribute to kernel updates.

24.7 Advantages of CNNs

CNNs:

- Scale efficiently to high-dimensional inputs
- Preserve spatial structure
- Achieve state-of-the-art performance in vision tasks

As a result, CNNs form the foundation of modern deep learning systems for image analysis.

25 PyTorch: Tensors, Data, and Gradients

25.1 Tensors

A tensor is a generalized multi-dimensional array:

- 0D: scalar
- 1D: vector
- 2D: matrix
- 3D+: higher-order tensors

Tensors form the core data structure in PyTorch.

25.2 GPU Acceleration

Tensor operations are optimized for parallel execution on GPUs, enabling large-scale neural network training.

25.3 Automatic Differentiation (Autograd)

PyTorch uses dynamic computation graphs. During the forward pass, operations are recorded in a Directed Acyclic Graph (DAG). Gradients are computed automatically using reverse-mode differentiation:

$$\frac{\partial \mathcal{L}}{\partial \theta}$$

25.4 Backward Pass

Calling:

```
loss.backward()
```

computes gradients for all tensors with `requires_grad=True`.

25.5 Gradient Control

- `requires_grad=True`: enables gradient tracking
- `torch.no_grad()`: disables gradient tracking during inference

26 Datasets and DataLoaders in PyTorch

26.1 Dataset Class

A dataset defines how individual samples are accessed and returned.

26.2 DataLoader

The DataLoader handles:

- Mini-batching
- Shuffling
- Parallel loading

This abstraction enables scalable and efficient training pipelines.

27 Deep Neural Network Compression Pipeline

27.1 Overview

The objective of this project is to reduce the storage and computational cost of neural networks while maintaining acceptable accuracy. This is achieved through a structured compression pipeline consisting of pruning, quantization, and entropy-based encoding.

Unlike traditional approaches, the current implementation operates on feature-based inputs and utilizes a Multi-Layer Perceptron (MLP) architecture. Convolutional layers are incorporated in the architecture but are not the primary focus at this stage.

The pipeline is modular and can be extended to more complex architectures such as CNNs in future work.

27.2 Pipeline Architecture

The complete workflow is given by:

Input Images → Feature Extraction → MLP Training → Pruning → Fine-Tuning → Quantization → Serialization
→ Compression

Each stage incrementally reduces model complexity while attempting to preserve predictive performance.

27.3 Model Architecture

The implemented model consists of convolutional feature extraction layers followed by fully connected layers:

- Convolutional layers for hierarchical feature extraction
- ReLU activations for non-linearity
- Max-pooling for spatial reduction
- Fully connected layers (MLP) for classification

The classifier operates on flattened feature representations.

27.4 Weight Pruning

Pruning removes redundant parameters by eliminating weights with small magnitudes.

Given weights w , pruning is defined as:

$$w' = \begin{cases} w, & |w| \geq \tau \\ 0, & |w| < \tau \end{cases}$$

where τ is a global threshold determined by the desired sparsity level.

Global pruning is applied by computing a threshold across all layers:

$$\tau = \text{percentile}(|w|, p)$$

This ensures uniform sparsity distribution across the network.

27.4.1 Effect of Pruning

- Reduces number of active parameters
- Introduces sparsity
- Slight degradation in accuracy

27.5 Weight Quantization

Quantization reduces the precision of weights by clustering them into a finite set of values.

Given weights w_i , quantization maps them to cluster centers:

$$w_i \rightarrow c_k, \quad k \in \{1, \dots, K\}$$

where K is the number of clusters.

K-Means clustering minimizes:

$$\sum_{i=1}^N \|w_i - c_{k(i)}\|^2$$

Instead of storing all weights, the model stores:

- Cluster indices (low memory)
- Cluster centers (small set)

27.5.1 Advantages

- Significant reduction in memory usage
- Exploits redundancy in weight values

27.6 Model Serialization

The compressed model is stored using a sparse representation. Weights are converted into Compressed Sparse Row (CSR) format:

- Non-zero values
- Column indices
- Row pointers

This reduces storage requirements for pruned models with high sparsity.

27.7 Entropy-Based Compression

After quantization, weight distributions become non-uniform. This enables further compression using entropy encoding techniques such as Huffman coding.

Symbols with higher frequency are assigned shorter binary codes, reducing overall storage size.

27.8 Training and Compression Strategy

The training pipeline follows:

1. Train baseline model
2. Apply global pruning

3. Fine-tune pruned model
4. Apply quantization
5. Evaluate compressed model

This sequential approach ensures minimal accuracy degradation.

27.9 Performance Metrics

The effectiveness of compression is evaluated using:

- Accuracy
- Model size (MB)
- Compression ratio
- Runtime memory usage

Compression ratio is defined as:

$$\text{Compression Ratio} = \frac{\text{Original Model Size}}{\text{Compressed Model Size}}$$

27.10 Observations

- Pruning introduces sparsity with minimal accuracy loss
- Quantization significantly reduces storage
- Combined compression yields substantial size reduction

The trade-off between accuracy and compression must be carefully balanced.

28 Results and Analysis

28.1 Baseline Model Performance

The model was initially trained without any compression techniques. The performance observed was:

- Training Accuracy: $\approx 99.43\%$
- Final Test Accuracy: **99.22%**

This serves as the reference performance for evaluating compression techniques.

28.2 Effect of Pruning

A global pruning strategy was applied with a target sparsity of 98%. The corresponding threshold was computed based on the weight distribution.

After pruning and fine-tuning:

- Final Test Accuracy: **98.59%**
- Achieved Sparsity: **98%**

Observation: Despite removing a large fraction of weights, the model retains high accuracy, demonstrating significant redundancy in neural network parameters.

28.3 Effect of Quantization

The pruned model was further quantized using K-Means clustering with $K = 4$ clusters.

After quantization and retraining:

- Final Test Accuracy: **99.22%**

Observation: Quantization did not degrade performance and in fact slightly improved accuracy due to regularization effects.

28.4 Compressed Model Performance

The compressed model was saved in sparse NPZ format and reloaded for evaluation.

- Compressed Model Accuracy: **98.25%**

Observation: A minor drop in accuracy is observed after full compression, which is expected due to aggressive parameter reduction.

28.5 Storage and Compression Analysis

Model	Size (MB)	Accuracy
Baseline (.pth)	1.6701	99.22%
Compressed (.npz)	1.1453	98.25%

The compression ratio achieved is:

$$\text{Compression Ratio} = \frac{1.6701}{1.1453} \approx \mathbf{1.46\times}$$

28.6 Memory Usage

- RAM usage: 1469.12 MB
- GPU Memory Allocated: 26.83 MB
- GPU Memory Reserved: 132.00 MB

Observation: The compressed model reduces storage requirements while maintaining manageable runtime memory usage.

28.7 Overall Insights

- Neural networks contain significant parameter redundancy
- Pruning achieves high sparsity with minimal accuracy loss
- Quantization effectively reduces precision without degrading performance
- Combined compression techniques provide substantial storage savings

The results demonstrate that efficient model compression is achievable without significantly compromising predictive capability.

29 Conclusion

This project presented a structured approach to deep neural network compression through a modular pipeline integrating pruning, quantization, and efficient model serialization. The implementation demonstrated how significant reductions in model size can be achieved without substantial loss in predictive performance.

Weight pruning was used to eliminate redundant parameters, introducing sparsity into the network while maintaining accuracy through fine-tuning. Quantization further reduced memory requirements by mapping continuous weights to a limited set of representative values, exploiting redundancy in learned parameters. The use of sparse storage formats enabled efficient serialization, and the overall pipeline highlighted how multiple compression techniques can be combined effectively.

An important aspect of this work is its modular design. The compression methods were implemented in a generalized manner, allowing seamless integration with different model architectures. Although the current implementation primarily focuses on MLP-based classification using extracted features, the framework is readily extensible to convolutional neural networks, which will be explored in future stages of the project.

From an experimental perspective, the project emphasizes the trade-off between model size, computational efficiency, and accuracy. While aggressive compression leads to higher storage savings, it may introduce performance degradation, necessitating careful tuning of sparsity levels and quantization parameters.

Overall, this work provides both a practical and theoretical foundation for neural network compression. It demonstrates how classical techniques such as sparsity and clustering can be leveraged to build efficient deep learning systems, and sets the stage for further exploration of advanced compression methods and real-world deployment scenarios.

30 Team Contributions

This project was carried out collaboratively, with each member contributing to different components of the system including model development, compression techniques, experimentation, and documentation.

Team Leader

Pranesh S (240771)

- Led overall project design and pipeline architecture
- Coordinated integration of pruning and quantization modules
- Oversaw experimentation, evaluation, and report structuring

Team Members

Arman Kumar Singh (240183)

- Worked on model implementation and training pipeline
- Assisted in debugging and performance tuning

Nipun Nandwani (240700)

- Contributed to pruning logic and sparsity analysis
- Helped evaluate compression-performance trade-offs

Uday Charan (240743)

- Implemented quantization methods and clustering logic
- Assisted in testing compressed model performance

Praveen Siddarth (240552)

- Worked on data handling and preprocessing pipeline
- Assisted in training and validation workflows

Sherwin Leo M.L (240970)

- Contributed to model evaluation and metric tracking
- Helped analyze runtime and memory usage

Vaibhav Gupta (241124)

- Assisted in implementation of compression modules
- Participated in debugging and testing

Sandeep Kumar Gupta (240928)

- Supported integration of different components
- Helped in experimentation and validation

Anubhav Jha (240152)

- Contributed to documentation and report preparation
- Assisted in organizing results and formatting